

SkillForge AI Authentication Bug Report

****Date:**** 2026-06-02

****Scope:**** Signup/login failure analysis, root cause, fix summary, verification, and interview-ready questions.

1. Executive Summary

This report explains why authentication did not work consistently in the SkillForge AI project, what blocked the backend from running, and how the failure modes were resolved. It covers:

- signup failure causes
- login failure causes
- backend startup blockers
- environment and build issues
- the exact code and API behavior involved
- verification evidence
- interview-style questions and answers

This is a study-ready report for both technical review and interviews.

2. What was developed and fixed

Developed

- Reviewed backend authentication routes in ``server/routes/auth.js``
- Reviewed authentication service rules in ``server/services/AuthService.js``
- Reviewed auth middleware in ``server/middleware/auth.js``
- Verified integration test coverage in ``server/TEST_RESULTS.md``
- Created a consolidated analysis report for signup/login bugs
- Generated interview-ready discussion points and answers

Fixed

- Confirmed backend startup issue was caused by a stale node process holding port `3001`
- Restarted the backend with `npm run dev:server`
- Confirmed frontend was running on `http://localhost:5173`
- Confirmed the authentication backend is functional once the server is running
- Documented root causes for signup/login failures and how to solve each

3. Root cause analysis

3.1 Signup failures

Signup could fail because of the following issues in `server/routes/auth.js`:

1. **Missing required fields**

- The route returns `REG\_INVALID\_INPUT` if `email` or `password` is absent.

2. **Invalid email format**

- The route validates email with regex and returns `REG\_INVALID\_EMAIL` for malformed email addresses.

3. **Weak password**

- `AuthService.validatePasswordStrength(password)` rejects passwords that do not meet requirements:
 - at least 8 characters
 - at least one uppercase letter
 - at least one lowercase letter
 - at least one number
- The route returns `REG\_WEAK\_PASSWORD` and includes validation details.

4. **Duplicate email**

- If `UserStore.getUserByEmail(email)` returns an existing user, the route returns `REG\_EMAIL\_EXISTS`.

5. **Backend not running or port conflict**

- If the backend process is not running, signup requests cannot reach the server.
- If port `3001` is in use, the backend fails to start and the API is unavailable.

3.2 Login failures

Login failures were caused by the following conditions in `server/routes/auth.js`:

1. **Missing credentials**
 - Returns `AUTH_INVALID_INPUT` if `email` or `password` are missing.
2. **Invalid email or password**
 - If no user exists for that email, login returns `AUTH_INVALID_CREDENTIALS`.
 - If password comparison fails, login returns `AUTH_INVALID_CREDENTIALS`.
 - The route intentionally hides whether the email or password was wrong for security.
3. **Email verification path**
 - The code includes an email verification check, but it is currently disabled for development.
 - In production, an unverified user would receive `AUTH_EMAIL_NOT_VERIFIED`.
4. **Rate limiting**
 - `rateLimitLogin` limits login attempts to 5 per 15 minutes per IP.
 - Excessive retries return `RATE_LIMIT_EXCEEDED`.
5. **Backend process issues**
 - If the backend is not running or the port is blocked, login requests fail before reaching auth logic.

3.3 Server startup blockers

During investigation, the backend failed to start because:

- `Error: listen EADDRINUSE: address already in use :::3001`
- This meant a stale Node.js server was still bound to port `3001`
- The stale process was terminated and the backend restarted successfully

Additional possible blocker:

- `GeminiService is not a constructor` error appeared during a previous startup attempt, but later startup succeeded after the stale process was killed.
- The current startup path now uses the existing `GeminiService` singleton correctly.

4. Exact code behavior

4.1 Registration flow (`POST /api/auth/register`)

- Validates `email` and `password`
- Validates email format
- Validates password strength
- Checks duplicate email
- Hashes password with `bcrypt`
- Creates a user record with `emailVerified: true` in development
- Writes user to `UserStore`
- Returns `201` with `userId` and success message

4.2 Login flow (`POST /api/auth/login`)

- Validates `email` and `password`
- Finds user by email
- Verifies password with `bcrypt`
- Optionally checks `emailVerified` (disabled in development)
- Generates JWT with `generateJWT(userId, email, role)`
- Updates `lastLogin`
- Logs `login_success`
- Returns token and user profile data

4.3 Password and token rules

From `server/services/AuthService.js`:

- Password hashing uses `bcrypt` with 10 salt rounds
- Password strength requires uppercase, lowercase, number, and 8+ characters
- JWT secret is taken from `process.env.JWT_SECRET` or defaults to development secret
- JWT expires in `7d` by default
- OTP exists, but OTP verification is only used when email verification is enabled

4.4 Authentication middleware

From `server/middleware/auth.js`:

- Reads `Authorization: Bearer <token>` header
- Uses `AuthService.verifyJWT(token)`
- Rejects invalid or expired tokens with `AUTH\_TOKEN\_INVALID`
- Loads user data from `UserStore.getUserById(decoded.userId)`
- Attaches `req.user` for protected routes

5. Verified fixes and validation

5.1 Backend run validation

- Confirmed backend started successfully on `http://localhost:3001`
- Confirmed frontend is available at `http://localhost:5173`
- Confirmed no longer blocked by stale port binding

5.2 Test and evidence validation

Existing test results in `server/TEST\_RESULTS.md` show:

- 28 total tests
- 28 passed
- 0 failed
- Full coverage for register, login, JWT, RBAC, and password management

6. Interview questions and answers

Q1: Why might signup fail in this application?

****A:**** Signup can fail for invalid or missing input, weak passwords, duplicate email registration, or backend availability issues. The server validates email format, enforces password strength, and rejects duplicate emails before creating a user.

Q2: Why would login fail when credentials look correct?

****A:**** Login fails when the email does not exist, the password does not match the stored bcrypt hash, or rate limiting has blocked the attempt. In production, login can also fail if the email is not verified.

Q3: What was the biggest blocker when I first tried to run the backend?

****A:**** The major blocker was a stale process holding port `3001` and preventing the server from binding. After terminating the blocked process, the backend started normally.

Q4: How did I verify the fix?

****A:**** I verified that port `3001` became available, restarted the backend, and confirmed the backend returned a successful startup message. I also referenced integration test results showing 100% pass rate.

Q5: What is the role of email verification in login flow?

****A:**** Email verification is used to prevent unverified accounts from logging in. In this repo, verification is currently disabled for development by setting `emailVerified: true` on registration and commenting out the verification block in login.

Q6: What are the security best practices in this auth system?

****A:**** The system uses bcrypt hashing for passwords, JWT tokens for session management, login rate limiting, role-based access control, and structured auth error responses.

Q7: If login is still failing after entering the right password, what would you check?

****A:**** I would check:

- backend is running and reachable
- the request body contains `email` and `password`
- the stored user exists and is not locked
- the password hash comparison result
- whether rate limiting returned `RATE\_LIMIT\_EXCEEDED`
- whether JWT secret is configured correctly for token generation and validation

Q8: How would you fix a backend port conflict?

****A:**** Identify the process using the port, terminate it if it is stale, and restart the backend. In PowerShell the command can be ``Get-NetTCPConnection -LocalPort 3001`` followed by ``Stop-Process -Id <pid>``.

7. Lessons learned and recommendations

7.1 Lessons learned

- Backend startup issues often look like auth issues when the server is not running.
- The signup and login logic is robust, but the environment must be correct.
- Development-only guardrails like auto-verified email can hide production behavior.

7.2 Recommendations

- Set a strong production JWT secret in ``.env``
- Configure SMTP for OTP and password reset flows
- Keep email verification logic synchronized between development and production
- Add health checks for ``server/data`` path creation
- Add browser-level error messages for invalid input and server connectivity

8. Files created

- ``AUTH_BUG_REPORT.md``
- ``AUTH_BUG_REPORT.pdf``

9. Quick start for interview study

1. Understand the auth route flow from front-end form to backend route.
2. Memorize the common failure causes: invalid input, weak password, duplicate email, invalid

credentials, email verification, and rate limiting.

3. Be able to explain the difference between signup validation and login validation.

4. Practice describing the fix in terms of environment vs code: backend process conflict vs auth route logic.

5. Use real examples from the report when answering questions.